Benchmarking Classical and Deep Forecasting Models

Multi-Horizon Walk-Forward Evaluation of Seasonal Sales and Multivariate Bike Demand

Aanshi Arora | Anish Saha | Harshit Rana

2026-08-08

Abstract

We compare SeasonalNaive, SARIMA, ARIMAX, LSTM, GRU, CNN-LSTM, Transformer, Transformer+PE on a short monthly champagne-sales series and a larger multivariate bike-demand series. All models are evaluated on both datasets using multiple non-overlapping test blocks and a common target set within each dataset. Neural models are fitted separately for each horizon and repeated over 3 seeds. The lowest pooled mean MAEs are as follows: Champagne: SARIMA at $h = 1$ (MAE 635.74), SeasonalNaive at $h = 3$ (MAE 641.46), SeasonalNaive at $h = 6$ (MAE 641.46), SeasonalNaive at $h = 12$ (MAE 641.46). Bike: GRU at $h = 1$ (MAE 31.75 $\pm$ 1.87), GRU at $h = 6$ (MAE 52.52 $\pm$ 1.00), LSTM at $h = 24$ (MAE 67.10 $\pm$ 0.81), SeasonalNaive at $h = 48$ (MAE 74.90). These are descriptive rankings under the supplied protocol, not universal architecture rankings. A fitting-after-origin issue affects early multi-step targets but does not affect the one-step results through this mechanism; its effect size has not been measured. Unequal fitting histories, oracle covariates and limited test coverage also restrict interpretation.

Keywords: forecasting; walk-forward evaluation; seasonal models; recurrent networks; positional embeddings; predictive accuracy.

1 Introduction

Short-horizon demand forecasting supports urban mobility fleet management, energy dispatch, retail inventory planning and transport scheduling. These applications often involve a target observed over time, seasonal cycles and contextual variables whose effects may be nonlinear. The practical question is whether a more flexible model improves forecasting enough to justify its additional complexity.

The classical framework of Box and Jenkins [1] addresses temporal dependence with the ARIMA family. SARIMA adds seasonal differencing and seasonal autoregressive and moving-average terms, while ARIMAX combines regression on explanatory variables with ARIMA errors. These models make their temporal and regression structure explicit, but the configurations considered here represent linear relationships. Recurrent networks such as LSTM [2] and GRU [3] instead learn nonlinear mappings from sequential inputs. CNN-LSTM hybrids place convolutional layers before recurrent processing to combine local feature extraction with sequential aggregation, a combination also studied in speech recognition [11]. Transformer encoders use multi-head self-attention to combine information across input positions [4].

These architectures differ in their *inductive bias*: the assumptions built into their structure before learning from data. Seasonal statistical models explicitly represent selected seasonal lags; gated recurrent networks process observations sequentially with learned memory; convolutional-recurrent models extract local patterns before recurrent aggregation; and attention encoders learn interactions among input positions. Such differences motivate an empirical comparison, but they do not determine a universal ranking. Performance also depends on the data, forecast horizon, available information, optimization and evaluation protocol.

This study compares all listed configurations on both a monthly seasonal sales series and a larger multivariate bike-demand series, using several temporal test blocks and direct multi-horizon neural forecasts. Its contribution is a documented benchmark with seed variability, a positional-information ablation and explicit evaluation limitations. Differences between the datasets are considered together rather than interpreted as isolated evidence of a data-volume or seasonality threshold for model selection.

1.1 Research questions

RQ1. At which horizons, and on which dataset, do the tested neural configurations have lower mean errors than the statistical and seasonal-naive baselines?

RQ2. Does the CNN-LSTM hybrid improve short-horizon accuracy relative to the pure recurrent models, and what parameter trade-off is observed between GRU and LSTM?

RQ3. What do the contrasting configurations and the positional-embedding ablation suggest about temporal inductive bias, within the limits of these datasets and this protocol?

2 Related work

Classical forecasting and strong baselines. ARIMA remains a standard framework for forecasting [1]. The M4 competition found strong performance from hybrid and combination approaches and highlighted the importance of statistical benchmarks [10]. This motivates treating classical methods and seasonal repetition as serious comparison points rather than assuming that greater neural complexity must yield better forecasts. The present study is much smaller than a forecasting competition and makes no claim of equivalent coverage.

Gated recurrent networks. LSTM introduced a memory mechanism for recurrent sequence learning [2]. The modern implementation used here has input, forget and output gates and a separate cell state. GRU uses reset and update gates without a separate LSTM-style cell state [3]. Chung et al. [9] found GRU and LSTM comparable on the sequence-modeling tasks they studied; that finding motivates comparison rather than establishing which model should win in this experiment. Parameter budgets are measured directly for the supplied configurations.

Convolutional-recurrent hybrids. Sainath et al. [11] combined convolutional, recurrent and dense processing for speech recognition. Their work provides background for combining complementary representations, not evidence that the present CNN-LSTM implementation must improve demand forecasting. Here the hybrid is tested empirically against recurrent and non-recurrent alternatives.

Transformers for time series. Self-attention [4] has been adapted to forecasting through approaches including Informer [12], Autoformer [13] and PatchTST [14]. Zeng et al. [15] reported that simple linear models outperformed the Transformer-based long-horizon configurations they evaluated. These contrasting findings motivate testing attention rather than treating its success in other domains as a guarantee of forecasting performance. None of these specialized forecasting architectures is implemented in the current benchmark.

Positional information and evaluation. The tested Transformer is a compact encoder followed by global average pooling. Without a positional signal, its inference output is invariant to reordering the input rows; the comparison with learned positional embeddings therefore probes an explicit architectural difference. Adam is the neural optimizer [5]. Predictive-accuracy comparisons are motivated by Diebold and Mariano [6], but the supplied pooled, seed-averaged procedure is described as DM-type because its selection and aggregation choices require additional inferential caution.

3 Datasets

The descriptive facts below are copied from the experiment summary. They describe the full datasets, not only fitting prefixes, and are not themselves model inputs.

- champagne: 105 months, 1964-01 – 1972-09; min 1,413, max 13,916
- champagne monthly means: highest December (10,821), lowest August (1,727)
- bike: 17,379 records, 2011-01-01 – 2012-12-31; calendar span 17,544 h => 165 hours missing
- bike top-3 mean-demand hours: 17:00 (461.5), 18:00 (425.5), 8:00 (359.0)

- bike mean hourly demand by year: 2011: 143.8, 2012: 234.7

Champagne provenance. The Perrin Frères series is commonly distributed through the Time Series Data Library; we cite the library's maintained package [8]. The exact primary collection source and the supplied CSV's measurement units have not been established. Accordingly, errors are reported in source-file sales units, not dollars or bottle counts. The repository citation establishes distribution through TSDL, not verified primary collection provenance.

Bike provenance and features. Bike demand is the UCI Bike Sharing target `cnt` [7]. The code uses `holiday`, `workingday`, `weathersit`, `temp`, `atemp`, `hum` and `windspeed`; historical target values form an additional neural input channel. It does not use `casual` or `registered`, nor explicit hour-of-day or cyclic calendar features. The file is ordered by `instant`, and timestamps combine date and hour. Missing timestamps are not inserted: horizons and seasonal lags therefore advance over records. We use "nominal hours" only as a shorthand; a record horizon can span more elapsed hours where gaps occur. Missing timestamps are different from missing entries within a recorded row.

4 Methodology

4.1 Walk-forward protocol

Each fold holds out a recent test block following its training prefix. Later folds expand the neural prefix and may use observations evaluated in previous folds. The champagne blocks are adjacent, whereas the bike blocks are spread across the later part of the series. Evaluation pools 48 champagne targets and 840 bike targets at each horizon. Horizons are 1/3/6/12 months for champagne and 1/6/24/48 records for bike. The evaluation blocks are listed below.

Table 1: Chronological evaluation blocks. Bike statistical models fit only the latest 720 prefix records; neural histories expand.

Dataset	Fold	Prefix length	Test period
Champagne	1	57	1968-10 to 1969-09
Champagne	2	69	1969-10 to 1970-09
Champagne	3	81	1970-10 to 1971-09
Champagne	4	93	1971-10 to 1972-09
Bike	1	9,558	2012-02-08 to 2012-02-15
Bike	2	11,472	2012-04-28 to 2012-05-05
Bike	3	13,385	2012-07-16 to 2012-07-23
Bike	4	15,298	2012-10-04 to 2012-10-11
Bike	5	17,211	2012-12-24 to 2012-12-31

Models are fitted once for each fold; neural models additionally have a separate fit for each horizon and seed. Within a test block, input histories and statistical filter states can incorporate observations that have become available at a later forecast origin, but model weights and parameters are not refitted. This is rolling-origin prediction rather than one fixed-origin forecast of the entire test block.

4.2 Neural preprocessing and targets

For input length L, feature count F and horizon h, a sample targeting time t is

$$X_t = (\mathbf{x}_{t-h-L+1}, \ldots, \mathbf{x}_{t-h}) \in \mathbb{R}^{L \times F}, \qquad \text{target } y_t.$$

Here the origin is $t - h$: the latest observation in the input window. If b is the first test index, candidate fitting targets range from $L + h - 1$ through $b - 1$, and test targets range from b to the end of the block. Thus the same test targets are used at every horizon. Each neural model directly predicts one scalar target; it does not recursively feed earlier predictions back into the input.

Feature minima and ranges come from rows before b. A zero range is replaced by one, and forecasts are inverse-transformed before metrics are calculated. The chronological tail of candidate fitting windows supplies validation; the earlier windows are shuffled only during mini-batch optimization. Scaling is computed before withholding validation, so its statistics include the validation prefix. No purge is applied between

optimization and validation windows; their historical inputs can overlap, and multi-step optimization labels can extend past the earliest validation origin. These are disclosed design limitations rather than changes made to the supplied run.

Fitting-after-origin qualification. Although test-block values do not enter training arrays, each fold's fitting prefix ends at $b-1$, which is later than origin $t-h$ for its first $h-1$ targets; weights, scaling statistics or statistical parameters can therefore use post-origin observations even though the target itself is excluded. The affected counts are champagne: 0/2/5/11 of 12 per block, for horizons 1/3/6/12; bike: 0/5/23/47 of 168 per block, for horizons 1/6/24/48; one-step evaluation is unaffected by this timing mechanism, SeasonalNaive fits nothing, and the effect size was not measured. Perturbing every test-block and later observation left neural training arrays and scaling statistics unchanged, while classical estimation samples were verified by code inspection without running classical fits here; these checks establish separation from test-block values, not complete forecast-origin purity.

4.3 Model configurations

SeasonalNaive. The forecast is $\hat{y}_t = y_{t-s}$, with an annual lag for champagne and a weekly-record lag for bike as specified in the code. The lag is at least as long as every evaluated horizon, so this observation is available at the origin. The model has no fitted weights, and its predictions are identical across the evaluated horizons for a given target.

SARIMA. The implementation uses SARIMA$(1,1,1)(0,1,1)_s$, with $s = 12$ for champagne and $s = 24$ for bike. Lowercase orders describe nonseasonal autoregression, differencing and moving-average terms; uppercase orders describe their seasonal counterparts. Orders are manually specified rather than selected in a tuning search. The bike baseline uses daily seasonality, while SeasonalNaive uses a weekly lag.

ARIMAX. The implementation uses ARIMA$(2,1,2)$ errors with external regressors and *no seasonal ARIMA term*. It uses the bike covariates at the forecast target time, including realized weather: this is an oracle-covariate benchmark, not a historical-only deployment forecast. Champagne instead uses deterministic sine and cosine annual Fourier terms with the harmonic count recorded in the summary. These are calendar-derived regressors rather than observed external measurements and can be known in advance.

Statistical estimation and prediction. The code fits maximum likelihood with stationarity and invertibility restrictions disabled and a fixed iteration limit. Champagne uses the entire prefix; bike SARIMA and ARIMAX use only the latest 720 fitting records. Parameters are estimated on the pre-test sample, then reused by `apply` on the combined fitting/test sequence. Each multi-step call uses dynamic prediction beginning just after its origin; the resulting path does not condition on observations after that origin. The separate parameter-fitting timing issue remains as described above.

LSTM and GRU. Each has a single recurrent layer of width 64, dropout probability 0.2 and a scalar dense output. The neural input includes only history through the origin, unlike target-time bike ARIMAX covariates.

CNN-LSTM. The model applies Conv1D with 32 filters, kernel size 3 and ReLU, max pooling of size 2, then Conv1D with 64 filters and the same kernel/activation and another pooling layer. LSTM(64), dropout(0.2) and a scalar dense output follow. The padding differs between datasets as reported in the training settings.

Transformer variants. Input features are projected to width 32. A single block uses four attention heads with key dimension 16 per head, residual addition and layer normalization, followed by a feed-forward block of widths 64 and 32 with another residual normalization. Global average pooling, dropout(0.2) and a scalar output complete the network. Transformer+PE adds a learned $L \times 32$ embedding after projection. The plain encoder has no explicit position signal: its attention block is permutation equivariant, and averaging makes its inference output invariant to reordering the input rows. This is an architectural property, not a measured accuracy result.

4.4 Optimization

Neural models use MSE loss, Adam at learning rate 10^{-3}, and early stopping on validation loss with best-weight restoration. Seeds are 42, 43, 44; reported variation is across these 3 runs. The settings and trainable parameter counts are recorded below. Parameter counts are not training-time, energy or latency measurements.

Table 2: Settings in model order LSTM, GRU, CNN-LSTM, Transformer, Transformer+PE.

Setting	Champagne	Bike
Window lengths	12/12/12/12/12	24/24/48/48/48
Batch size	4	128
Epoch cap	300	60
Patience	20	5
Validation fraction	0.2	0.1
Convolution padding	same	valid

Table 3: Trainable parameters counted by the framework.

model	champagne (F=1)	bike (F=8)
LSTM	16,961	18,753
GRU	12,929	14,273
CNN-LSTM	39,425	40,097
Transformer	12,833	13,057
Transformer+PE	13,217	14,593

4.5 Metrics and aggregation

For errors $e_t = y_t - \hat{y}_t$ on n targets,

$$\mathrm{MAE} = n^{-1} \sum_t |e_t|, \qquad \mathrm{RMSE} = \sqrt{n^{-1} \sum_t e_t^2},$$

$$\mathrm{MAPE} = 100 n^{-1} \sum_t \frac{|e_t|}{|y_t|}, \qquad \mathrm{sMAPE} = 100 n^{-1} \sum_t \frac{2|e_t|}{|y_t| + |\hat{y}_t|}.$$

The supplied code uses these expressions without clipping denominators or predictions. MAPE can become large when bike counts are small and should not be the primary basis for comparison.

Predictions are concatenated across folds separately for each seed; metrics are then computed for that pooled series. Neural entries report the mean and sample standard deviation of the pooled per-seed metrics, not the metrics of an averaged forecast. The code uses sample SD with `ddof=1`. Seed SD measures sensitivity to training randomness, not uncertainty across future periods or a confidence interval. Per-fold MAE tables average MAE over seeds and describe temporal variation separately. Saved forecasts are rounded before pooled metrics are recomputed; the build script checks consistency at the summary's displayed precision.

4.6 DM-type comparisons

For each dataset/horizon, the script chooses the model with lowest mean pooled MAE and compares it only with the other models. Neural predictions are averaged across seeds *before* forming the errors for these tests. Thus their comparison unit differs from the main metric tables.

For absolute or squared loss, let d_t be selected-model loss minus comparator loss. The code estimates the variance of the mean using Bartlett-weighted autocovariances with bandwidth

$$q = \min\{\max(h - 1, \lfloor n^{1/3} \rfloor), n - 2\},$$

and divides the mean loss difference by its estimated standard error. It uses a two-sided t_{n-1} reference distribution, with no multiple-comparison correction. Negative statistics favor the selected model for that particular loss. We use a nominal 0.05 threshold only for describing the supplied exploratory comparisons, not as a guarantee of superiority. Pooling also treats fold boundaries and irregular record gaps as adjacent positions. The accompanying summary provides all comparisons; a printed zero p-value reflects rounding, not exact zero.

5 Results

5.1 Champagne sales

Table 4: Champagne: pooled metrics at h=1. Neural entries show mean and sample standard deviation across seeds.

Model	MAE	RMSE	MAPE (%)	sMAPE (%)
SeasonalNaive	641.46	862.46	13.56	13.37
SARIMA	635.74	868.96	15.26	13.92
ARIMAX	1196.64	1602.65	27.31	23.20
LSTM	1436.34 ± 29.29	1960.29 ± 67.28	31.23 ± 0.10	29.15 ± 1.39
GRU	1280.90 ± 71.71	1842.87 ± 121.10	29.18 ± 1.70	26.49 ± 1.32
CNN-LSTM	1046.71 ± 206.05	1547.36 ± 391.53	22.40 ± 4.58	20.75 ± 4.06
Transformer	2228.90 ± 51.02	3194.43 ± 53.26	47.16 ± 0.75	39.76 ± 1.08
Transformer+PE	1063.66 ± 77.93	1588.16 ± 231.51	23.01 ± 1.58	21.33 ± 1.75

Table 5: Champagne: mean MAE by horizon. Neural entries show mean and sample standard deviation across seeds.

Model	h=1	h=3	h=6	h=12
SeasonalNaive	641.46	641.46	641.46	641.46
SARIMA	635.74	707.83	723.91	670.93
ARIMAX	1196.64	1191.17	1195.45	1148.03
LSTM	1436.34 ± 29.29	1848.71 ± 20.42	1713.35 ± 60.15	1363.17 ± 186.43
GRU	1280.90 ± 71.71	1830.56 ± 91.40	1237.03 ± 34.51	868.32 ± 31.69
CNN-LSTM	1046.71 ± 206.05	753.64 ± 70.45	898.13 ± 159.84	717.83 ± 51.54
Transformer	2228.90 ± 51.02	2109.30 ± 8.23	2058.03 ± 17.44	2077.99 ± 16.81
Transformer+PE	1063.66 ± 77.93	1045.74 ± 156.58	896.42 ± 144.71	1278.23 ± 489.67

At $h = 1$, SARIMA has the lowest pooled mean MAE (635.74); the lowest neural mean is CNN-LSTM (1046.71 ± 206.05).

At $h = 3$, SeasonalNaive has the lowest pooled mean MAE (641.46); the lowest neural mean is CNN-LSTM (753.64 ± 70.45).

At $h = 6$, SeasonalNaive has the lowest pooled mean MAE (641.46); the lowest neural mean is Transformer+PE (896.42 ± 144.71).

At $h = 12$, SeasonalNaive has the lowest pooled mean MAE (641.46); the lowest neural mean is CNN-LSTM (717.83 ± 51.54).

Transformer+PE has lower mean MAE than the plain Transformer at every evaluated horizon; this is a result for the supplied encoder, not all forecasting Transformers.

The one-step SARIMA–SeasonalNaive absolute-loss test gives $p = 0.913$, so it does not establish a predictive-accuracy difference at the nominal threshold. Failure to reject does not prove equivalence.

No fitted model has lower mean MAE than SeasonalNaive at the longer horizons. This statement concerns MAE and these test blocks.

5.2 Bike demand

Table 6: Bike: pooled metrics at h=1. Neural entries show mean and sample standard deviation across seeds.

Model	MAE	RMSE	MAPE (%)	sMAPE (%)
SeasonalNaive	74.90	127.47	126.00	50.57
SARIMA	48.28	75.20	66.05	49.96
ARIMAX	63.91	95.33	180.62	53.38
LSTM	33.91 ± 2.52	53.16 ± 5.60	57.81 ± 6.78	36.60 ± 2.42
GRU	31.75 ± 1.87	47.51 ± 4.01	58.42 ± 8.84	36.34 ± 2.75
CNN-LSTM	50.59 ± 1.00	70.67 ± 0.38	130.65 ± 2.53	50.50 ± 0.56
Transformer	123.98 ± 2.91	171.02 ± 1.10	530.68 ± 40.32	78.01 ± 2.00
Transformer+PE	37.71 ± 1.51	54.89 ± 3.06	89.91 ± 12.92	41.18 ± 1.46

Table 7: Bike: mean MAE by horizon. Neural entries show mean and sample standard deviation across seeds.

Model	h=1	h=6	h=24	h=48
SeasonalNaive	74.90	74.90	74.90	74.90
SARIMA	48.28	123.82	90.45	137.84
ARIMAX	63.91	131.01	131.74	134.47
LSTM	33.91 ± 2.52	53.72 ± 1.50	67.10 ± 0.81	78.17 ± 0.99
GRU	31.75 ± 1.87	52.52 ± 1.00	70.27 ± 2.17	80.27 ± 0.83
CNN-LSTM	50.59 ± 1.00	61.31 ± 2.22	78.79 ± 2.53	77.01 ± 1.94
Transformer	123.98 ± 2.91	126.13 ± 3.99	129.43 ± 2.60	128.50 ± 1.86
Transformer+PE	37.71 ± 1.51	64.16 ± 13.82	77.04 ± 14.00	79.45 ± 10.23

At $h = 1$, GRU has the lowest pooled mean MAE (31.75 ± 1.87); the lowest neural mean is GRU (31.75 ± 1.87).

At $h = 6$, GRU has the lowest pooled mean MAE (52.52 ± 1.00); the lowest neural mean is GRU (52.52 ± 1.00).

At $h = 24$, LSTM has the lowest pooled mean MAE (67.10 ± 0.81); the lowest neural mean is LSTM (67.10 ± 0.81).

At $h = 48$, SeasonalNaive has the lowest pooled mean MAE (74.90); the lowest neural mean is CNN-LSTM (77.01 ± 1.94).

Transformer+PE has lower mean MAE than the plain Transformer at every evaluated horizon; this is a result for the supplied encoder, not all forecasting Transformers.

The GRU–LSTM absolute-loss comparisons give $p = 0.041$ at the shortest horizon and $p = 0.348$ at the next horizon. These are unadjusted exploratory tests on seed-averaged forecasts, not tests of the per-seed mean metrics.

At the next-day nominal horizon, LSTM versus SeasonalNaive gives $p = 0.274$ for absolute loss; the lower tabulated LSTM MAE is not conclusive evidence of superiority.

At the longest bike horizon, CNN-LSTM has lower RMSE than SeasonalNaive (113.42 ± 2.28 versus 127.47), although SeasonalNaive has lower MAE. Rankings therefore depend on the loss metric.

Interpretation boundary. These are the supplied results, not corrected-origin rerun results. Rankings among fitted models at the longer champagne and bike horizons should be treated especially cautiously because fitting can use observations after some claimed origins. No adjusted errors or effect sizes are available.

6 Discussion

RQ1: observed rankings. The horizon-wise results answer RQ1 for the tested configurations. These rankings concern minimum mean MAE, not minimum error on every fold or every metric. The one-step

results avoid the disclosed fitting-after-origin mechanism, although the other information and training-history differences remain.

RQ2: convolution and parameter budget. The horizon-wise tables show that the hybrid's relative standing depends on dataset and horizon; they do not establish that convolution always improves recurrent forecasts. GRU has fewer trainable parameters than LSTM (champagne: 12,929 versus 16,961; bike: 14,273 versus 18,753). This supports a smaller parameter budget, not a measured speed advantage or guaranteed long-horizon stability. Differences in input-window lengths and all untuned settings remain confounded with architecture.

RQ3: positions and inductive bias. The position-aware encoder's lower mean MAEs are verified directly from the tables. *Hypothesis:* explicit positions help distinguish temporal arrangements that the unpositioned pooled encoder cannot distinguish. This is consistent with the architectural property, but the experiment does not isolate every cause: positions add parameters, and optimization varies. *Hypothesis:* the small seasonal sales series favors simple seasonal structure, while recurrent memory is useful for short-horizon bike demand. These are explanations to investigate, not demonstrated causal mechanisms.

Timing-bias hypothesis. Post-origin fitting information could plausibly favor fitted models relative to the unfitted seasonal baseline. If it did, observations that fitted models still did not beat SeasonalNaive in mean MAE could be conservative. This is only a hypothesis: the direction and magnitude were not measured, and additional information need not improve every fit. It cannot establish how rankings would change in an origin-aligned experiment.

No universal selection rule. The datasets differ simultaneously in size, sampling frequency, covariates, seasonality and evaluation coverage. The observations therefore cannot identify an optimal architecture by data volume or seasonality alone. They justify retaining simple baselines and evaluating candidate configurations under the intended forecast conditions.

7 Limitations

- **Fitting after some origins.** For the first $h-1$ targets per fold, fitted weights or statistical parameters use observations later than the nominal origin, despite excluding test-block targets. One-step evaluation and SeasonalNaive are unaffected by this mechanism. The effect size is unmeasured; longer-horizon fitted-model rankings and the conservative-baseline hypothesis require caution.
- **Convergence.** Champagne SARIMA did not converge in its first fold. Its supplied forecasts remain in the reported results; an optimizer flag does not quantify their accuracy impact.
- **Epoch caps.** Bike runs reaching the 60-epoch cap are LSTM: 3/60, GRU: 5/60, CNN-LSTM: 0/60, Transformer: 25/60, Transformer+PE: 14/60. Transformer variants may be under-trained, but cap hits alone do not prove that longer training would improve held-out accuracy.
- **Unequal histories.** Bike statistical models fit 720 records, whereas neural pre-test histories range from 9,558–17,211 records before withholding validation. A model-family effect cannot be separated from this difference.
- **Unequal information and seasonality.** Bike ARIMAX has oracle future covariates and no seasonal ARIMA term. SeasonalNaive and SARIMA use different bike seasonal lags, and neural models use historical covariates and varying windows. These are configuration comparisons.
- **Validation and irregular records.** Scaling includes the validation prefix; no horizon-aware label purge is used. Missing bike hours are not imputed, so nominal-hour horizons represent records and may differ in elapsed duration.
- **Exploratory tests.** DM-type tests use seed-averaged forecasts, select the best comparator from evaluated data, compare best-versus-others only, and lack multiplicity correction. Their pooled variance calculation does not respect fold boundaries or time gaps. A large p-value is not proof of equivalence.

- **Limited repetitions and tuning.** Only 3 seeds are used. Hyperparameters and statistical orders are untuned, and some architecture choices differ beyond the model family. There is no controlled runtime or scalability measurement.
- **Limited coverage and metrics.** There are few test blocks, including just 48 pooled champagne targets. Bike MAPE is unreliable near low counts. Differences between datasets are confounded, ruling out general architecture-selection rules.

8 Conclusion

The supplied results favor different configurations across datasets and horizons, as reported in the main tables. The positional ablation and parameter counts provide additional descriptive evidence for the tested configurations. The paper does not establish a universal winner, a runtime advantage, or leakage-free multi-step evaluation. Its defensible contribution is a documented comparison with transparent uncertainty, full reported metrics and an evidence-checked generation process. Future work could align fitting to each forecast origin, match histories and covariate availability, improve validation, and expand temporal coverage; none of those changes has been run for this paper.

Supplementary results

The accompanying summary and metric export provide full horizon-wise MAE, RMSE, MAPE and sMAPE, per-fold MAE, candidate training-window counts, optimization diagnostics and all best-versus-others DM-type comparisons. They complement the principal tables in this manuscript.

9 Reproducibility

The reported numbers come from the supplied complete-run summary and metric export; model specifications follow the accompanying experiment code. Recorded environment: python 3.13.15, numpy 2.1.3, pandas 2.2.3, tensorflow 2.20.0, statsmodels 0.15.0, GPU True. Neural seeds are 42, 43, 44. The report generation checks exported metrics, rankings and numerical comparisons against the source files. DM statistics are reproduced from the summary rather than independently regenerated from per-target forecasts. No experiments were rerun for this report.

References

[1] G. E. P. Box and G. M. Jenkins. *Time Series Analysis: Forecasting and Control*. Holden-Day, 1970.

[2] S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997. https://doi.org/10.1162/neco.1997.9.8.1735.

[3] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk and Y. Bengio. Learning phrase representations using RNN encoder–decoder for statistical machine translation. *EMNLP*, pp. 1724–1734, 2014. https://aclanthology.org/D14-1179/.

[4] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser and I. Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 2017. https://arxiv.org/abs/1706.03762.

[5] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. *ICLR*, 2015. Preprint first posted in 2014. https://arxiv.org/abs/1412.6980.

[6] F. X. Diebold and R. S. Mariano. Comparing predictive accuracy. *Journal of Business & Economic Statistics*, 13(3):253–263, 1995. https://doi.org/10.1080/07350015.1995.10524599.

[7] H. Fanaee-T. Bike Sharing [Dataset]. UCI Machine Learning Repository, 2013. https://doi.org/10.24432/C5W894.

[8] R. Hyndman and Y. Yang. *tsdl: Time Series Data Library*, 2018. Maintained package documentation: https://pkg.yangzhuoranyang.com/tsdl/. This is a repository citation; exact primary champagne-series provenance remains to be confirmed.

[9] J. Chung, C. Gulcehre, K. Cho and Y. Bengio. Empirical evaluation of gated recurrent neural networks on sequence modeling. NIPS Deep Learning and Representation Learning Workshop, 2014. https://arxiv.org/abs/1412.3555.

[10] S. Makridakis, E. Spiliotis and V. Assimakopoulos. The M4 Competition: Results, findings, conclusion and way forward. *International Journal of Forecasting*, 34(4):802–808, 2018. https://doi.org/10.1016/j.ijforecast.2018.06.001.

[11] T. N. Sainath, O. Vinyals, A. Senior and H. Sak. Convolutional, long short-term memory, fully connected deep neural networks. *ICASSP*, 2015. https://doi.org/10.1109/ICASSP.2015.7178838.

[12] H. Zhou, S. Zhang, J. Peng, S. Zhang, J. Li, H. Xiong and W. Zhang. Informer: Beyond efficient Transformer for long sequence time-series forecasting. *AAAI*, 2021. https://arxiv.org/abs/2012.07436.

[13] H. Wu, J. Xu, J. Wang and M. Long. Autoformer: Decomposition Transformers with auto-correlation for long-term series forecasting. *Advances in Neural Information Processing Systems*, 2021. https://arxiv.org/abs/2106.13008.

[14] Y. Nie, N. H. Nguyen, P. Sinthong and J. Kalagnanam. A time series is worth 64 words: Long-term forecasting with Transformers. *ICLR*, 2023. https://arxiv.org/abs/2211.14730.

[15] A. Zeng, M. Chen, L. Zhang and Q. Xu. Are Transformers effective for time series forecasting? *AAAI*, 2023. https://arxiv.org/abs/2205.13504.